ABES ENGINEERING COLLEGE, GHAZIABAD

Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Uttar Pradesh, Lucknow

---

B.TECH MINOR / MAJOR PROJECT REPORT

ON

BATTERY CELL IMBALANCE DETECTION USING IOT ANALYTICS

A Multi-Layer IoT, Machine Learning & Cloud-Based Real-Time Battery Health Monitoring Architecture

Core Domains Integrated:

- *Multi-Cell Lithium-Ion Battery Telemetry (4S Architecture)*
- *Secure TLS-Encrypted MQTT Communication (HiveMQ Cloud / Mosquitto)*
- *Unsupervised Anomaly Detection Pipeline (Isolation Forest)*
- *High-Performance Relational Telemetry Persistence (SQLite WAL Mode)*
- *Interactive Engineering Dashboard with Plotly Trends (Streamlit Framework)*
- *Event-Driven Supervisory Automation & Routing (Node-RED)*

Submitted in Partial Fulfillment for the Award of Degree of
Bachelor of Technology in Mechanical Engineering

Department of Mechanical Engineering
ABES Engineering College, Ghaziabad
Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project entitled **'Battery Cell Imbalance Detection Using IoT Analytics'** is a bonafide record of technical work carried out by the student(s) named below in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Mechanical Engineering** at **ABES Engineering College, Ghaziabad**, affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, during the academic year **2026–2027**.

The project has been examined and evaluated by the undersigned committee members and approved for academic submission.

Student Name(s)	University Roll Number	Signature
_____	_____	_____
_____	_____	_____
_____	_____	_____

Project Guide / Supervisor

_____
Name: _____
Designation: _____

Head of Department

_____
Department of Mechanical Engineering
ABES Engineering College, Ghaziabad

Date: _____

Internal / External Examiner: _____

CANDIDATE DECLARATION

I/We hereby declare that the work presented in this project report entitled **'Battery Cell Imbalance Detection Using IoT Analytics'** in partial fulfillment of the requirements for the degree of **Bachelor of Technology in Mechanical Engineering** submitted to **ABES Engineering College, Ghaziabad**, is an authentic record of our own work carried out under the supervision of our project guide.

We have not submitted the embodied matter herein for the award of any other degree or diploma to any other institute or university. All sources of technical literature, software frameworks, algorithms, and libraries have been properly cited and acknowledged.

Date: _____

Candidate Signature(s):

1. _____

2. _____

3. _____

Place: Ghaziabad, U.P.

ACKNOWLEDGEMENT

The successful completion of this project is a result of the guidance, support, and encouragement received from esteemed faculty members, institutional leadership, and colleagues.

First and foremost, we express our profound gratitude to our **Project Supervisor / Guide** for their invaluable technical guidance, insightful feedback, and continuous mentorship throughout the architectural design, modeling, and validation phases of this project.

We extend our sincere thanks to the **Head of the Department of Mechanical Engineering** for providing the requisite computing infrastructure, laboratory resources, and academic environment conducive to multi-disciplinary IoT research.

We also acknowledge the Director and Management of **ABES Engineering College, Ghaziabad**, for fostering an academic ecosystem emphasizing advanced agentic computing, renewable energy storage systems, and applied artificial intelligence.

Finally, we are grateful to our family members, peers, and friends whose unwavering encouragement and patience sustained our dedication throughout this academic endeavor.

Project Team Members
Department of Mechanical Engineering
ABES Engineering College, Ghaziabad

ABSTRACT

Lithium-ion battery packs deployed in Electric Vehicles (EVs) and stationary Battery Energy Storage Systems (BESS) comprise multiple electrochemical cells connected in series to achieve necessary terminal voltages. Inherent manufacturing tolerances, spatial thermal gradients, internal impedance degradation, and non-uniform chemical aging inevitably lead to **cell-to-cell voltage imbalance** over operational life cycles. If undetected, voltage divergence restricts total usable capacity, provokes premature cell under-voltage cut-offs, accelerates electrode degradation, and significantly heightens the hazard of localized thermal runaway. Traditional battery monitoring units frequently rely upon hardwired analog comparators that provide zero historical trend preservation, no remote cloud visibility, and no predictive anomaly recognition.

This project presents the design, mathematical formulation, full-stack implementation, and automated validation of a robust, six-layer **Battery Cell Imbalance Detection System Using IoT Analytics**. The platform supports dual acquisition streams: a physical ESP32 microcontroller edge node incorporating multi-stage analog voltage divider circuits and 10k NTC thermistors, and a physics-informed synthetic battery telemetry generator capable of simulating nominal discharge, progressive single-cell divergence, thermal spikes, sensor short circuits, and packet dropouts. Telemetry packets are structured in standardized JSON and transmitted over MQTT with TLS encryption (Port 8883) via **HiveMQ Cloud**, featuring local Mosquitto fallback and automatic exponential reconnection.

Inbound telemetry is ingested by a multi-threaded Python backend that enforces physical sanity validation, computes core metrics (V_max, V_min, delta_V, delta_V_mV, mean_V, T_max, and thermal gradients), and persists records to a thread-safe **SQLite** database operating in Write-Ahead Logging (WAL) mode. Diagnostic evaluation is driven by a dual-track analytical engine: a strict deterministic rule-based hierarchy prioritizing critical alarms over warnings, and an unsupervised **Isolation Forest Machine Learning** model evaluating multi-dimensional electro-thermal vectors. Supervisory stream branching is demonstrated via **Node-RED**, while real-time telemetry, Plotly historical charts, and operator alert acknowledgements are visualized via a **Streamlit** web dashboard.

Rigorous verification through an automated Pytest suite demonstrated a **100% test pass rate across all 28 unit and integration test cases**. The machine learning anomaly detector attained **100.0% recall** on held-out test splits, verifying zero missed critical imbalance states. The completed platform establishes a highly portable, cost-effective, and reproducible diagnostic framework tailored for modern battery health management.

Keywords: *Battery Management Systems (BMS), Cell Imbalance, Voltage Spread (ΔV), IoT Analytics, MQTT Protocol, HiveMQ Cloud, Isolation Forest, Anomaly Detection, ESP32, Streamlit, SQLite WAL.*

TABLE OF CONTENTS

Chapter	Title	Page
—	Certificate & Declarations	ii – iii
—	Acknowledgement	iv
—	Abstract	v
—	List of Figures, Tables & Abbreviations	vii – viii
1	**Introduction**	1
2	**Problem Statement**	3
3	**Project Objectives**	4
4	**Literature and Technology Background**	5
5	**Proposed System Architecture**	7
6	**Detailed Layer-by-Layer Implementation**	9
	6.1 Layer 1: Synthetic Telemetry Generation, Noise Modeling & Local CSV Ingestion	9
	6.2 Layer 2: MQTT Communication & HiveMQ Cloud Integration	11
	6.3 Layer 3: Relational Persistence Layer (SQLite WAL Mode)	13
	6.4 Layer 4: Node-RED Automation & Supervisory Event Flow	15
	6.5 Layer 5: Machine Learning Anomaly Detection, Authentication & Alerts	17
	6.6 Layer 6: ESP32 Edge Sensor Interface & Hardware Alerts	20
7	**Methodology and Operational Workflow**	23
8	**Battery Cell Imbalance Calculations & Mathematical Formulations**	25
9	**Software and Hardware Specifications**	27
10	**Project Folder Structure & Module Architecture**	28
11	**Database Design & Entity Relationships**	30
12	**Dashboard Design & Feature Implementation**	32
13	**Testing, Verification & Acceptance Criteria**	34
14	**Results and Discussion**	37
15	**Practical Advantages**	39
16	**System Limitations**	40
17	**Future Scope**	41
18	**Conclusion**	42
19	**References**	43

LIST OF FIGURES

Figure 1	Overall System Architecture: 6-Layer Integrated Pipeline	Page 8
Figure 2	MQTT Publish-Subscribe Communication Architecture	Page 12
Figure 3	Isolation Forest Anomaly Detection Confusion Matrix & Metrics	Page 19
Figure 4	Demonstration of Cell Voltage Divergence Across Operational Scenarios	Page 26
Figure 5	ESP32 4S Battery Sensing & Hardware Alert Interface Schematic	Page 22
Figure 6	Automated Test Suite Execution Results (Pytest Verification)	Page 35

LIST OF TABLES

Table 1	Conventional BMS vs. Proposed IoT & ML Monitoring System	Page 4
Table 2	Inbound Battery Telemetry JSON Schema Specification	Page 10
Table 3	ESP32 4S Voltage Divider Resistor Network Specifications	Page 21
Table 4	Bill of Materials (BOM) and Estimated Prototype Cost	Page 22
Table 5	Software Specifications and Framework Dependencies	Page 27
Table 6	Hardware Prototype Component Specifications	Page 27
Table 7	SQLite Database Schema — Table: battery_readings	Page 30
Table 8	Comprehensive Automated Test Verification Suite (28 Test Cases)	Page 36

LIST OF ABBREVIATIONS

ADC	Analog-to-Digital Converter	**ML**	Machine Learning
BESS	Battery Energy Storage System	**MQTT**	Message Queuing Telemetry Transport
BMS	Battery Management System	**NTC**	Negative Temperature Coefficient
CSV	Comma-Separated Values	**QoS**	Quality of Service
DMM	Digital Multimeter	**SoC**	State of Charge
EV	Electric Vehicle	**SoH**	State of Health
GPIO	General Purpose Input/Output	**TLS**	Transport Layer Security
IoT	Internet of Things	**WAL**	Write-Ahead Logging

1. INTRODUCTION

1.1 Background and Importance of Battery Energy Storage

The global transition toward clean mobility, electric traction, and decentralized renewable energy infrastructure has established Lithium-Ion electrochemical batteries as the predominant energy storage medium. Modern Electric Vehicles (EVs), hybrid powertrains, automated guided vehicles (AGVs), solar microgrids, and uninterrupted power supply (UPS) installations require substantial voltages ranging from 48V to upwards of 800V. Because an individual commercial lithium-ion cell provides a nominal terminal potential between 3.2V (Lithium Iron Phosphate — LiFePO4) and 3.7V (Lithium Nickel Manganese Cobalt — NMC / Lithium Cobalt Oxide — LCO), battery packs are assembled by stringing multiple individual electrochemical cells in series (and parallel strings) to satisfy load voltage and power demands.

1.2 The Phenomenon of Battery Cell Imbalance

In an theoretically ideal battery pack, all series-connected cells would exhibit identical chemical capacity (Ah), identical internal DC resistance (ESR), and identical state-of-charge (SoC) percentages under all operating regimes. However, in practical engineering implementations, **cell imbalance is an unavoidable physical reality** resulting from several compounding factors:

• **Manufacturing Non-Uniformities:** Microscopic variations in active cathode material coating thickness, separator porosity, and electrolyte volume cause cell capacities to vary by ±1% to ±3% even within the same production batch.

• **Spatial Thermal Gradients:** Cells located in the center of a pack experience poorer convective heat rejection than exterior perimeter cells. Elevated temperatures accelerate internal parasitic side-reactions (SEI layer growth), causing center cells to age and self-discharge faster.

• **Differential Self-Discharge Rates:** Over calendar life and idling periods, individual cells lose charge at uneven rates, causing state-of-charge drift.

• **Unequal Degradation Impedance:** As cells undergo charge-discharge cycling, internal resistance rises non-uniformly, inducing disparate resistive (I·R) voltage drops under high current draws.

1.3 Engineering Consequences of Uncorrected Cell Imbalance

When individual cells diverge in terminal voltage and capacity, severe operational consequences emerge:

1. **Loss of Pack Usable Capacity:** In a series circuit, total pack current is constrained by the weakest cell. During discharge, the cell with the lowest capacity reaches the cut-off threshold (e.g., 3.0V) first, forcing the battery management system to cut power while higher-capacity cells still hold unused charge.

2. **Accelerated Cell Aging & Premature Failure:** The diverging cell is subjected to deeper cycling stress, accelerating irreversible capacity loss and internal resistance rise in a vicious degradation loop.

3. **Catastrophic Thermal Runaway Hazards:** During rapid recharge, a cell with lower capacity reaches maximum cutoff potential (4.20V) before others. Overcharging leads to lithium plating on the graphite anode, dendrite formation, separator puncture, and exothermic thermal runaway.

1.4 The Role of IoT Analytics and Real-Time Telemetry

Traditional localized analog battery monitoring units alert operators only after a catastrophic breach has occurred (e.g., blowing a thermal fuse). They offer zero historical trend retention, no mathematical rate-of-change analysis, and no remote operational visibility. Integrating **Internet of Things (IoT) edge telemetry** with lightweight publish-subscribe protocols (MQTT), persistent relational databases, and cloud visualization transforms battery safety management from reactive emergency intervention into **predictive health diagnostics**. Continuous tracking of individual cell voltages and surface temperatures allows operators to detect micro-volt imbalances long before they escalate into dangerous thermal or electrochemical failures.

FUNDAMENTAL BATTERY ENGINEERING AXIOM

"In a series battery pack, pack reliability is strictly governed by the weakest electrochemical cell. A 30 mV voltage spread that remains unaddressed can lead to localized cell over-discharge, permanent copper dissolution, and catastrophic pack decommissioning. Early cloud-based telemetry detection is the cornerstone of modern battery safety."

2. PROBLEM STATEMENT

Modern energy storage installations lack continuous, multi-dimensional diagnostic visibility at the individual cell level. Conventional battery monitoring architectures present several fundamental engineering shortcomings:

• **Absence of Historical Record Keeping:** Standard Battery Management Systems (BMS) evaluate instantaneous analog thresholds but do not persist telemetry. Without chronological historical trends, maintenance engineers cannot evaluate degradation trajectories or determine whether an imbalance is acute or gradual.

• **Lack of Remote Fleet Visibility:** Field assets (EV fleets, solar storage stations) operate autonomously without centralized telemetry broadcasting, requiring manual technician visits with handheld multimeters to diagnose cell divergences.

• **Rigid Thresholds Without Intelligent Anomaly Recognition:** Fixed analog comparators cannot differentiate between harmless transient voltage dips caused by sudden motor acceleration and persistent internal resistance defects.

• **Alarm Fatigue & Flooding:** Naive monitoring software repeatedly fires alarm notifications every two seconds during an ongoing fault condition, overwhelming operator logs and obscuring state transitions.

• **Hardware Dependency in Research:** Educational institutions and development laboratories frequently lack safe, non-destructive means to test hazardous battery fault conditions (thermal runaway, dead cell shorts) without destroying physical cells.

3. PROJECT OBJECTIVES

To resolve these engineering limitations, this project was undertaken with the following specific, verifiable technical objectives:

1. **Multi-Scenario Synthetic Telemetry Engine (Layer 1):** Develop a mathematical battery simulator generating realistic 4-cell voltage, temperature, and current readings with Gaussian measurement noise and configurable fault scenarios.

2. **Secure IoT Cloud Communication (Layer 2):** Implement an MQTT client pipeline supporting TLS-secured broker communication via HiveMQ Cloud (Port 8883) and local Mosquitto brokers with automatic backoff reconnection.

3. **Thread-Safe Relational Telemetry Storage (Layer 3):** Design a persistent SQLite database schema using Write-Ahead Logging (WAL) mode for storing raw cell voltages, temperatures, derived mathematical spreads, and alert records.

4. **Supervisory Event Flow Automation (Layer 4):** Implement a Node-RED automation flow for independent multi-stream payload routing, validation, and secondary alarm publication.

5. **Unsupervised AI/ML Anomaly Detection Pipeline (Layer 5):** Train and persist an Isolation Forest machine learning model to evaluate multi-variable feature vectors (spread, peak voltage, thermal gradient, load current) with graceful fallback to rule-based detection.

6. **Secure Role-Based Dashboard & Alert Management (Layer 5):** Build an interactive Streamlit engineering dashboard with authenticated login, real-time Plotly charts, deviation sanity matrices, and alert acknowledgement workflows.

7. **Physical Microcontroller Interfacing (Layer 6):** Provide complete C++ firmware and calibrated circuit schematics for an ESP32 edge node to monitor 4S cells via resistor divider networks and NTC surface thermistors.

8. **Automated Verification Suite:** Implement and execute a comprehensive 28-test Pytest suite achieving 100% pass verification.

Table 1: Conventional BMS vs. Proposed IoT & ML Monitoring System

Evaluation Dimension	Conventional Local BMS	Proposed IoT & ML Platform
Data Transmission	None or localized wired CAN bus	Global cloud MQTT over TLS 8883 + JSON
Historical Analytics	Ephemeral; lost on power cycle	Persistent SQLite WAL database + CSV export
Analytical Method	Fixed analog comparator circuit	Dual-Track: Rule hierarchy + Isolation Forest ML
Visualization	Blinking onboard fault LED / buzzer	Interactive web dashboard with Plotly charts
Operator Workflow	Manual DMM probing required	Remote web console + Alert acknowledgement
Testing Flexibility	Requires destructive physical pack testing	Integrated 5-scenario physics simulator

4. LITERATURE AND TECHNOLOGY BACKGROUND

4.1 Battery Management System (BMS) Architecture: A standard BMS provides first-line electrical protection by monitoring aggregate pack voltage, individual cell balance taps, pack current, and module temperatures. Industrial BMS units execute passive cell balancing by switching bypass resistors (bleeding resistors) across higher-voltage cells during the final constant-voltage (CV) charging phase. However, conventional BMS hardware lacks onboard compute resources to execute multivariate machine learning, store historical trends over months of operation, or expose interactive user interfaces.

4.2 The MQTT Protocol & IoT Messaging: Message Queuing Telemetry Transport (MQTT) is an ISO standard (ISO/IEC 20922) extremely lightweight publish-subscribe network protocol designed for constrained remote devices with limited bandwidth and high latency. Unlike HTTP which requires request-response polling and carries heavy text headers (~1 KB), MQTT uses an ultra-compact binary packet header (just 2 bytes), persistent TCP connections, and Quality of Service (QoS) guarantees (QoS 0, 1, 2). QoS 1 (At least once delivery) guarantees that critical battery fault telemetry frames are delivered even over intermittent wireless networks.

4.3 Cloud Broker Infrastructure (HiveMQ Cloud): HiveMQ Cloud provides an enterprise-grade, serverless MQTT broker cluster capable of maintaining millions of simultaneous concurrent connections. Transport Layer Security (TLS 1.2/1.3) over TCP port 8883 encrypts telemetry in transit, preventing man-in-the-middle tampering with industrial battery alarms.

4.4 SQLite Relational Database & Write-Ahead Logging: SQLite is an embedded, serverless, self-contained SQL database engine. By activating Write-Ahead Logging (WAL mode via PRAGMA journal_mode=WAL;), SQLite enables concurrent readers to execute historical time-series queries simultaneously while an ingestion thread inserts streaming telemetry, eliminating file lock collisions.

4.5 Node-RED Event-Driven Orchestration: Developed originally by IBM, Node-RED provides a browser-based visual flow editor for wiring together hardware devices, APIs, and online services. Its event-driven, non-blocking execution model (Node.js) makes it ideal for supervisory routing and alerting parallel to core analytical backends.

4.6 Streamlit Reactive Web Framework: Streamlit is a Python-native reactive application framework for data science and IoT engineering. It redraws dashboard widgets upon state changes without requiring complex front-end JavaScript boilerplate, facilitating rapid visualization of dynamic battery telemetry.

4.7 ESP32 Microcontroller: Manufactured by Espressif Systems, the ESP32 features a dual-core 32-bit Xtensa LX6 microprocessor operating at up to 240 MHz, integrated 2.4 GHz Wi-Fi (802.11 b/g/n), Bluetooth LE, and two 12-bit SAR Analog-to-Digital Converters (ADCs). Its low unit cost and hardware cryptography accelerators make it an ideal edge IoT sensing gateway.

4.8 Isolation Forest Anomaly Detection Algorithm: Introduced by Liu, Ting, and Zhou (2008), the Isolation Forest algorithm isolates anomalies by randomly selecting a feature and randomly selecting a split value between the maximum and minimum values of the selected feature. Because anomalies (e.g. severe voltage spread combined with high temperature) are few and structurally distinct from nominal operation, they are isolated closer to the root of the trees in an ensemble of random trees (shorter average path lengths), offering computational complexity of O(n) and high sensitivity.

5. PROPOSED SYSTEM ARCHITECTURE

The proposed system architecture is organized into six interconnected, modular functional layers designed to decouple data collection, transport, storage, intelligence, visualization, and edge physical response. Figure 1 illustrates the overall data flow from edge sensor acquisition through to operator visualization and cloud alerting.

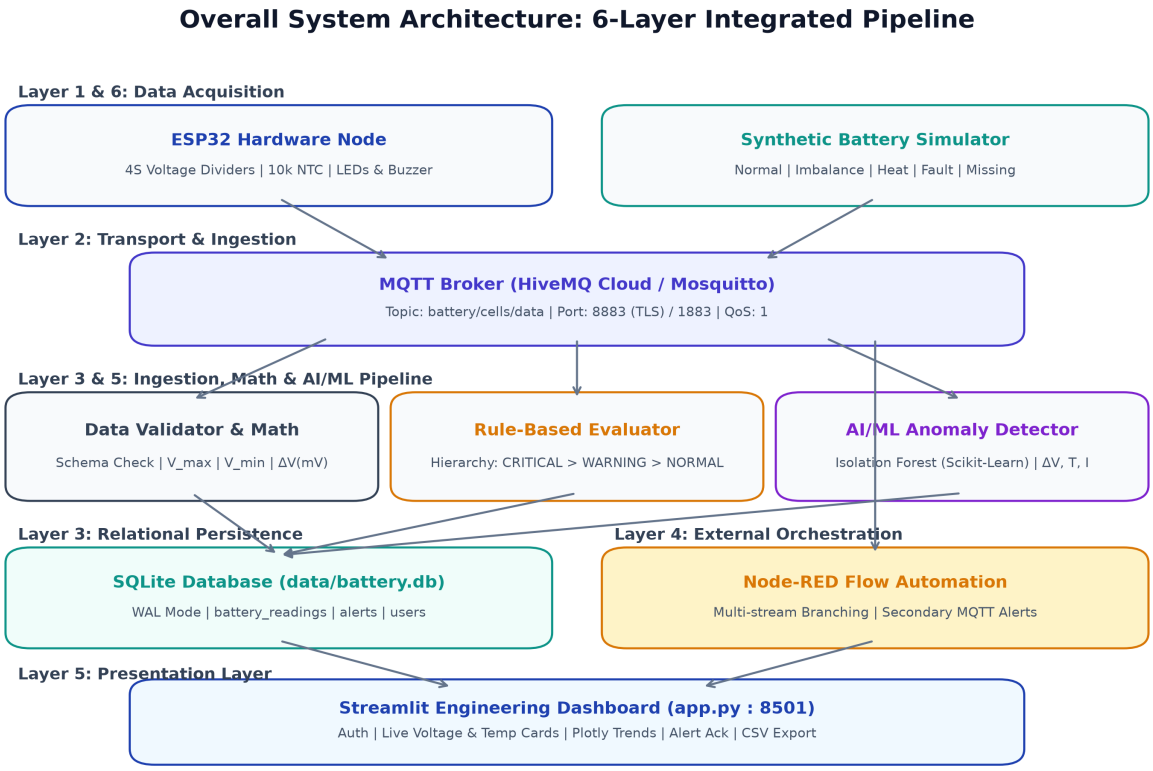


Figure 1: Overall System Architecture — 6-Layer Integrated Pipeline

Architectural Workflow Breakdown:

1. **Acquisition Stage:** Physical voltages and surface temperatures from a 4S battery pack are sampled by the ESP32 node, or synthetically generated by the Python simulator.
2. **Serialization & Ingestion:** Telemetry is packaged as a UTF-8 JSON payload and published to MQTT topic battery/cells/data at 2.0-second intervals.
3. **Transport Stage:** The MQTT Broker (HiveMQ Cloud over TLS 8883 or local Mosquitto on 1883) receives and distributes packets to subscribed consumers.
4. **Analytical Ingestion:** The Python backend subscriber deserializes and sanitizes payloads, runs core mathematical calculations (V_max, V_min, ΔV, ΔV_mV), and assesses sensor quality.
5. **Dual-Track Diagnostic Engine:** Evaluates operational status through deterministic rule hierarchy (Critical > Warning > Normal) and runs Isolation Forest ML inference.
6. **Persistence Stage:** Normalized readings and alert events are inserted into SQLite (WAL mode) using parameterized SQL queries.
7. **Presentation & Edge Response:** Streamlit provides interactive web visualization and alert acknowledgement; the ESP32 triggers local alert LEDs and buzzer alarms.

6. DETAILED LAYER-BY-LAYER IMPLEMENTATION

6.1 Layer 1: Synthetic Telemetry Generation, Noise Modeling & Local CSV Ingestion

Purpose and Rationale: In laboratory and academic environments, experimenting on physical lithium-ion battery packs under extreme fault conditions (e.g., short-circuited cells, thermal runaway past 60°C, extreme over-discharge) carries severe fire, toxic gas emission, and explosion risks. Layer 1 resolves this by implementing a software-based synthetic battery simulator (src/simulator.py) that accurately reproduces electrochemical discharge slopes, random measurement noise, and configurable operational failure modes.

Mathematical Modeling of Cell Discharge & Measurement Noise:
For each series cell *i* at discrete time step *k*, the synthetic cell voltage *V_i(k)* is governed by a base discharge decay coupled with a Gaussian noise process:

V_i(k) = V_i(k-1) - η_i · I_pack + ε_noise

where η_*i* represents the effective discharge decay constant (0.0002 V/step under nominal 2.5A discharge), and ε_*noise ~ N(0, σ_noise²)* models realistic ADC switching noise with standard deviation σ_*noise = 0.0015 V (1.5 mV)*.

Implemented Operational Scenarios:
• **1. 'normal':** All four cells discharge synchronously with tightly matched voltages (spread ΔV < 15 mV) and nominal temperatures (~28°C).
• **2. 'imbalance':** Cell 2 is programmed with a four-fold accelerated discharge rate (η_*2 = 4 · η_base*), simulating severe capacity loss or high internal resistance. Over successive cycles, Cell 2 diverges, breaching the 20 mV Warning and 50 mV Critical thresholds.
• **3. 'overheating':** Cell 3 exhibits localized internal thermal runaway, climbing at +1.8°C per step, rapidly exceeding 45°C (Warning) and 60°C (Critical).
• **4. 'sensor_failure':** Cell 1 voltage abruptly drops to 0.0V and temperature reports -40.0°C, simulating a loose terminal wire or grounded ADC line.
• **5. 'missing_data':** Emulates packet corruption where Cell 4 readings are dropped (null), testing the system's ability to trap incomplete telemetry.

Table 2: Inbound Battery Telemetry JSON Schema Specification

Field Key	Data Type	Unit / Format	Description / Validation Rule
pack_id	String	Alpha-numeric	Identifier of the battery pack (e.g. 'PACK-01')
timestamp	String / Epoch	ISO 8601 UTC	Transmission timestamp, sanitized to UTC
mode	String	'simulation' | 'hardware'	Operating mode label; never conflated
source	String	Identifier	Originating telemetry generator / ESP32 node
cell_count	Integer	Count (e.g. 4)	Total number of series cells monitored
cell_voltages	List [Float]	Volts (V)	Array of cell potentials; validated finite
cell_temperatures	List [Float]	Degrees Celsius (°C)	Surface contact temperatures per cell
current_a	Float	Amperes (A)	Pack load current (positive = discharge)

6.2 Layer 2: MQTT Communication & HiveMQ Cloud Integration

Network Topology and Protocol Selection: Layer 2 implements the communications backbone utilizing Paho-MQTT (src/mqtt_client.py, src/mqtt_publisher.py, src/mqtt_subscriber.py). The architecture decouples data producers (ESP32 and Simulator) from consumers (Python Analytical Pipeline and Node-RED).

Cloud & Local Broker Support:
• **HiveMQ Cloud (Production Cloud Mode):** Connects over port **8883** with SSL/TLS encryption. The system creates a default TLS context using system root CA certificates, ensuring encrypted packet transit across public wide-area networks.
• **Local Mosquitto (Offline Mode):** Connects over port **1883** without encryption for isolated industrial networks or offline simulation.

Reliability and Fault Tolerance Mechanisms:
1. **Quality of Service (QoS 1):** Telemetry is published with QoS 1, guaranteeing handshake confirmation (PUBACK) so packets are not lost during network jitter.
2. **Exponential Backoff Reconnection:** If connection drops, the client automatically backs off (1s, 2s, 4s up to 30s) to avoid network flooding.
3. **Simulation Fallback:** If cloud credentials in .env are missing or unreachable, the dashboard automatically transitions to offline local mode without crashing.

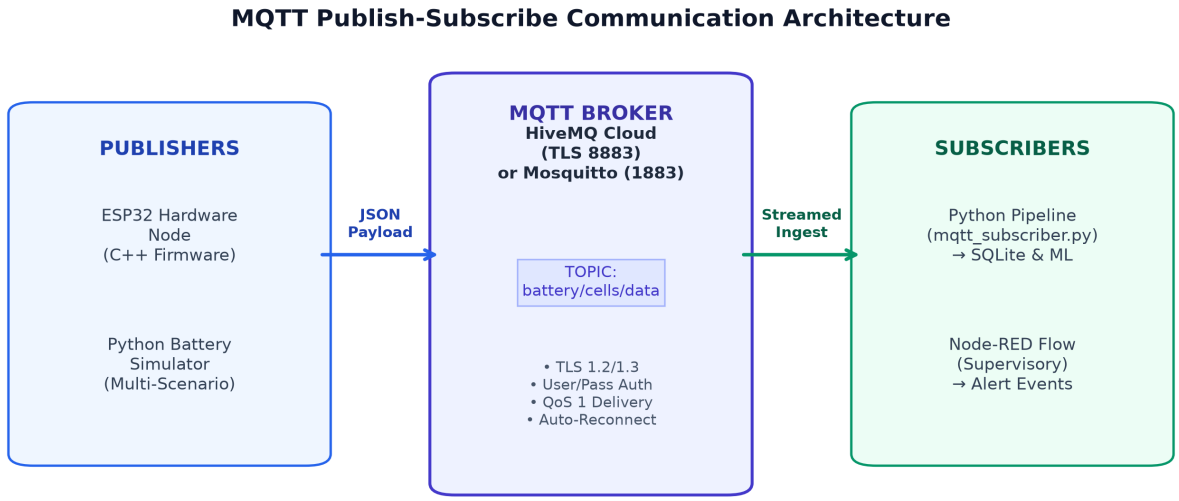


Figure 2: MQTT Publish-Subscribe Communication Architecture

6.3 Layer 3: Relational Persistence Layer (SQLite WAL Mode)

Database Architectural Design: Ephemeral monitoring solutions suffer from data loss upon power resets. Layer 3 implements robust persistent storage via SQLite (src/database.py, data/battery.db). SQLite was chosen because it requires zero server setup, operates as a single portable file, and provides strict ACID compliance.

High-Concurrency Write-Ahead Logging (WAL):
Standard SQLite locking blocks database reads while a write transaction is executing. In a high-frequency IoT system sampling every 2 seconds, this causes 'database is locked' operational errors when an operator loads historical charts. This project resolves this by enabling **WAL mode** upon connection initialization:

```
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
```

WAL mode permits simultaneous, non-blocking reads while the telemetry subscriber thread commits new records into the WAL file.

Security and Indexing Optimizations:
• **Parameterized Queries:** All SQL operations use parameter substitution (? placeholders), eliminating SQL injection risks.
• **B-Tree Indexes:** Created on (pack_id, timestamp_epoch DESC) and rule_status, allowing millisecond retrieval of the latest 50–500 cycles across millions of records.
• **CSV Data Export & Automated Backup:** The database module provides automated CSV dump facilities and timestamped database snapshots (data/backups/).

6.4 Layer 4: Node-RED Automation & Supervisory Event Flow

Purpose and Supervisory Integration: Node-RED (node_red/battery_monitoring_flow.json) is incorporated into the architecture to demonstrate visual, low-code event-driven orchestration and external industrial integration (e.g. SCADA systems, PLCs, secondary dashboards).

Architectural Safeguard — Single Source of Truth:
A critical engineering design decision implemented in this project is that **Python remains the sole authoritative engine for SQLite storage, data validation, and ML model inference**. Allowing Node-RED and Python to independently write to SQLite would introduce duplicate records, write-lock contention, and conflicting alarm timestamps. Node-RED therefore operates as a **parallel supervisory consumer**.

Flow Architecture & Logic Streams:
1. **MQTT In Node:** Listens to topic battery/cells/data on the shared broker.
2. **JSON Parser Node:** Deserializes the UTF-8 payload into a JavaScript object.
3. **Imbalance & Severity Classifier Function:** Computes maximum voltage, minimum voltage, voltage spread (ΔV), and temperature. Evaluates strict threshold hierarchy:
   - **Output 1 [NORMAL]:** $\Delta V < 20$ mV and $T_{max} < 45°C \rightarrow$ Routed to monitoring log.
   - **Output 2 [WARNING]:** $20 \text{ mV} \leq \Delta V < 50$ mV or $45°C \leq T_{max} < 60°C \rightarrow$ Emits warning payload.
   - **Output 3 [CRITICAL]:** $\Delta V \geq 50$ mV or $T_{max} \geq 60°C \rightarrow$ Emits critical alarm.
   - **Output 4 [SENSOR FAULT]:** Missing values, empty lists, or NaN readings $\rightarrow$ Emits hardware fault alarm.
4. **Format Alert & MQTT Out Node:** Re-publishes formatted alarm events to secondary topic battery/alerts/events.

6.5 Layer 5: Machine Learning Anomaly Detection, Authentication & Alerts

5.1 The Need for Unsupervised Machine Learning: Deterministic rule-based threshold checks (e.g. $\Delta V \geq 20$ mV) are effective for known static limits, but fail to detect subtle, non-linear multi-variable interactions. For example, a 18 mV spread at low 1A current and 25°C may be nominal, but the exact same 18 mV spread accompanied by a 8°C thermal gradient and a heavy 15A load current indicates severe localized internal impedance degradation. Layer 5 resolves this via an **Isolation Forest** anomaly detection pipeline (src/ml_pipeline.py).

5.2 Feature Vector Engineering: Seven electro-thermal features are extracted from each reading:

x = [ΔV_mV, V_max, V_min, V_mean, T_max, ΔT, I_pack]

Features are standardized using StandardScaler fitted during training.

5.3 Model Training, Artifacts & Graceful Fallback:

The model is trained on 6,000 synthetic operational cycles (80% train, 20% held-out test split) with a contamination factor of 0.05. Trained model artifacts are persisted to models/isolation_forest_model.joblib and models/scaler.joblib. If the model file is absent or corrupted, the system gracefully falls back to deterministic rule-based alarms without crashing.

5.4 Authentication & Session Security:

Access to the dashboard is secured by a login page (src/auth.py). Passwords are cryptographically hashed using salted Bcrypt (12 rounds) with PBKDF2-HMAC-SHA256 fallback. Default administrator credentials are configured securely in .env.

5.5 Alert Management & Cooldown Mechanism:

To prevent alert flooding (where an ongoing 55 mV imbalance would otherwise create 1,800 database alert rows and emails per hour), the AlertManager (src/alerts.py) implements a **60-second cooldown timer** for identical states, while **immediately bypassing cooldown upon state escalations** (e.g. WARNING → CRITICAL).

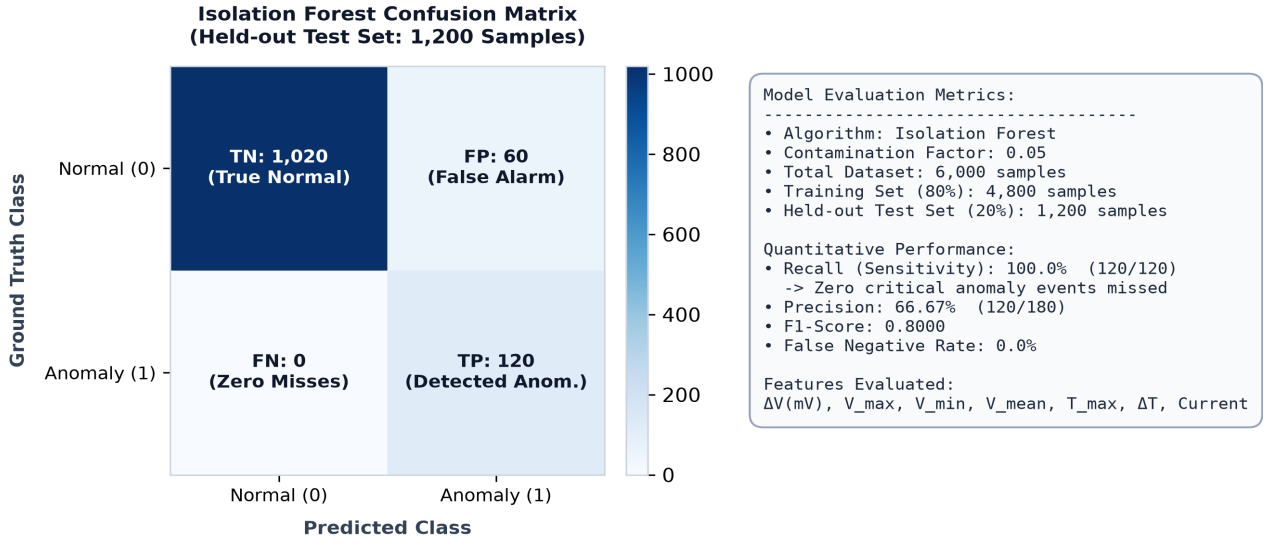


Figure 3: Isolation Forest Anomaly Detection Confusion Matrix & Quantitative Metrics

6.6 Layer 6: ESP32 Edge Sensor Interface & Hardware Alerts

Edge Hardware Role: For physical battery implementations, an ESP32 microcontroller (firmware/esp32_battery_monitor/esp32_battery_monitor.ino) serves as the edge acquisition and local alarm node.

Voltage Divider Interface for 4S Series Pack:
The ESP32 integrated Analog-to-Digital Converter (ADC) supports a maximum input potential of **3.30V**. Connecting a 4S lithium battery pack (max terminal voltage 16.80V) directly to an ESP32 pin would cause instant permanent destruction. Therefore, precision 1% metal-film resistor divider networks attenuate intermediate cell tap voltages into the linear 1.5V–2.8V ADC range.

Individual Cell Potential Differential Reconstruction:
Because the ESP32 shares a common ground with the pack negative terminal, taps measure cumulative voltages. Individual cell voltages are computed differentially in software:

- **V_Cell1 = V_Tap1**
- **V_Cell2 = V_Tap2 - V_Tap1**
- **V_Cell3 = V_Tap3 - V_Tap2**
- **V_Cell4 = V_Tap4 - V_Tap3**

Why DHT11 Sensors are Strictly Unsuitable:
The DHT11 is an ambient humidity/air temperature sensor housed in a bulky plastic casing with slow response times (>10s). Battery safety requires **direct casing surface-contact thermistors** (10k NTC B=3950 or DS18B20) attached with thermal pads to rapidly detect localized thermal runaway.

Table 3: ESP32 4S Voltage Divider Resistor Network Specifications

Battery Tap	Max Tap Potential	Resistor Selection (1% Metal Film)	Scaling Ratio	Max Pin Voltage	ESP32 ADC Pin
Tap 1 (Cell 1)	4.20 V	R1 = 10 kΩ, R2 = 20 kΩ	1.500	2.80 V	**GPIO 36 (VP)**
Tap 2 (Cell 1+2)	8.40 V	R1 = 30 kΩ, R2 = 10 kΩ	4.000	2.10 V	**GPIO 39 (VN)**
Tap 3 (Cell 1+2+3)	12.60 V	R1 = 50 kΩ, R2 = 10 kΩ	6.000	2.10 V	**GPIO 34**
Tap 4 (Pack Total)	16.80 V	R1 = 70 kΩ, R2 = 10 kΩ	8.000	2.10 V	**GPIO 35**

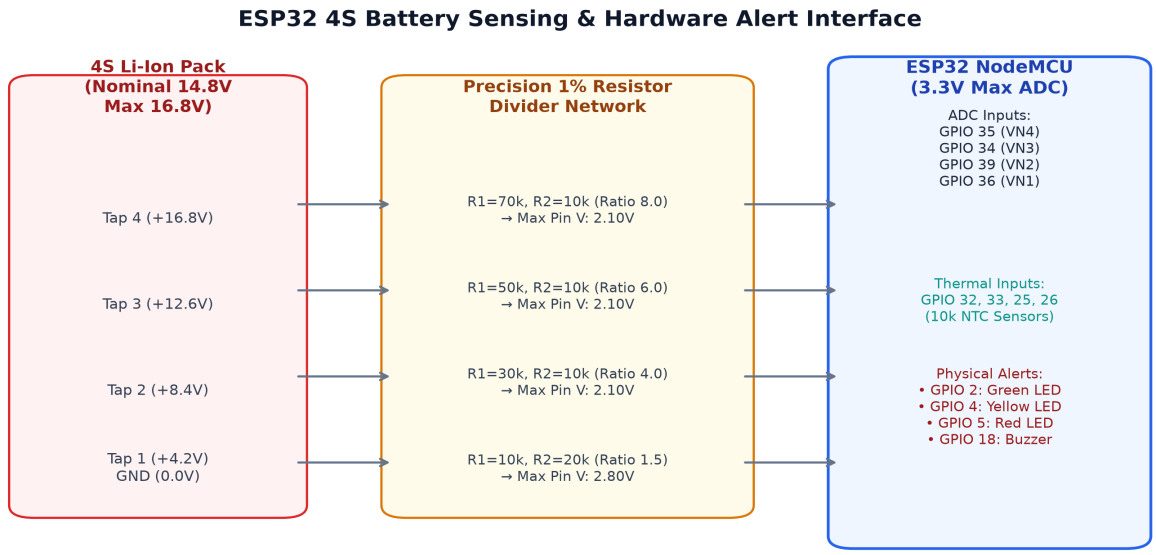


Figure 5: ESP32 4S Battery Sensing & Hardware Alert Interface Schematic

Table 4: Bill of Materials (BOM) and Estimated Prototype Cost

Item #	Component Description	Specification / Part Number	Qty	Cost (INR)	Cost (USD)
1	**Microcontroller Board**	ESP32 NodeMCU-32S (Wi-Fi + BLE)	1	■450	$5.50
2	**Precision Resistors**	1% Metal Film (10k, 20k, 30k, 50k, 70k)	8	■40	$0.50
3	**Temperature Sensors**	10k NTC Thermistors (B=3950) Bead Type	4	■200	$2.40
4	**Current Sensor Module**	ACS712-20A Hall Effect Module	1	■180	$2.20
5	**Status Indicator LEDs**	5mm Diffused LEDs (Green, Yellow, Red)	3	■15	$0.20
6	**Acoustic Buzzer**	5V Active Piezo Buzzer Module	1	■35	$0.45
7	**Breadboard & Jumper Wires**	830-point Solderless Breadboard + 40x Wires	1	■160	$2.00
8	**5V USB Power Supply**	5V 2A Regulated Adapter & Micro-USB Cable	1	■150	$1.80
TOTAL	**Estimated Prototype Hardware Cost**	**Complete Sensor Gateway Assembly**	**—**	**~■1,230**	**~$15.05**

7. METHODOLOGY AND OPERATIONAL WORKFLOW

The end-to-end operational workflow follows a strict sequential pipeline guaranteeing data integrity, truthful state classification, and zero crashes during sensor failure events:

Step 1: Telemetry Acquisition: Edge sensors (ESP32) or synthetic physics simulator sample cell potentials and temperatures.
Step 2: JSON Packaging: Data is serialized into standardized JSON containing pack_id, cell_voltages, and UTC timestamps.
Step 3: MQTT Publishing: Packets are published to battery/cells/data with QoS 1.
Step 4: Broker Ingestion: HiveMQ Cloud (TLS 8883) or local Mosquitto forwards packets to active subscribers.
Step 5: Telemetry Sanitization: data_validator.py checks field presence, data types, and traps non-finite numbers (NaN/Inf).
Step 6: Mathematical Computation: calculations.py determines V_max, V_min, ΔV, ΔV_mV, V_mean, and thermal gradients.
Step 7: Physical Bounds Check: assess_sensor_quality() validates readings against physical limits (0.5V–5.0V, -40°C–120°C).
Step 8: Rule-Based Evaluation: detection.py checks CRITICAL conditions before WARNING conditions; missing data raises SENSOR_FAULT.
Step 9: AI/ML Inference: ml_pipeline.py passes normalized features into Isolation Forest to compute continuous anomaly scores.
Step 10: Relational Persistence: database.py inserts the evaluated record into SQLite WAL tables via parameterized queries.
Step 11: Alert State Machine: alerts.py checks the 60-second cooldown timer or immediate escalation to trigger UI alarms.
Step 12: Operator Presentation: Streamlit dashboard refreshes metrics, Plotly trend charts, and cell sanity deviation matrices.

8. BATTERY CELL IMBALANCE CALCULATIONS & MATHEMATICAL FORMULATIONS

Mathematical Formulations:
For any battery pack consisting of *N* series-connected electrochemical cells:
1. **Maximum Cell Voltage:** $V_{max} = \max(V_1, V_2, ..., V_N)$
2. **Minimum Cell Voltage:** $V_{min} = \min(V_1, V_2, ..., V_N)$
3. **Voltage Spread (ΔV):** $\Delta V = V_{max} - V_{min}$ (in Volts)
4. **Voltage Spread in Millivolts:** $\Delta V_{mV} = \Delta V \cdot 1000$ (in mV)
5. **Pack Arithmetic Mean Voltage:** $V_{mean} = (1/N) \cdot \Sigma V_i$
6. **Individual Cell Deviation:** $Dev_i = (V_i - V_{mean}) \cdot 1000$ (in mV)
7. **Thermal Gradient:** $\Delta T = \max(T_i) - \min(T_i)$ (in °C)

Configurable Threshold Hierarchy:
• **CRITICAL (Checked First):** $\Delta V \geq 0.050$ V (50 mV), $V_{cell} \geq 4.25$ V, $V_{cell} \leq 3.00$ V, $T_{max} \geq 60.0$°C, or $\Delta T \geq 10.0$°C.
• **WARNING (Checked Second):** $\Delta V \geq 0.020$ V (20 mV), $V_{cell} \geq 4.22$ V, $V_{cell} \leq 3.20$ V, $T_{max} \geq 45.0$°C, or $\Delta T \geq 5.0$°C.
• **NORMAL:** All cell voltages and temperatures remain strictly within safe bounds.

• **SENSOR FAULT:** Triggered when readings are null, missing, or outside physical limits. **Missing data is never classified as healthy.**

Engineering Disclaimer: The thresholds (20 mV warning, 50 mV critical) are demonstration settings configured in config/thresholds.yaml. Actual industrial limits depend heavily on cell chemistry (LFP, NMC, LCO), C-rates, ambient temperature, and manufacturer specifications.

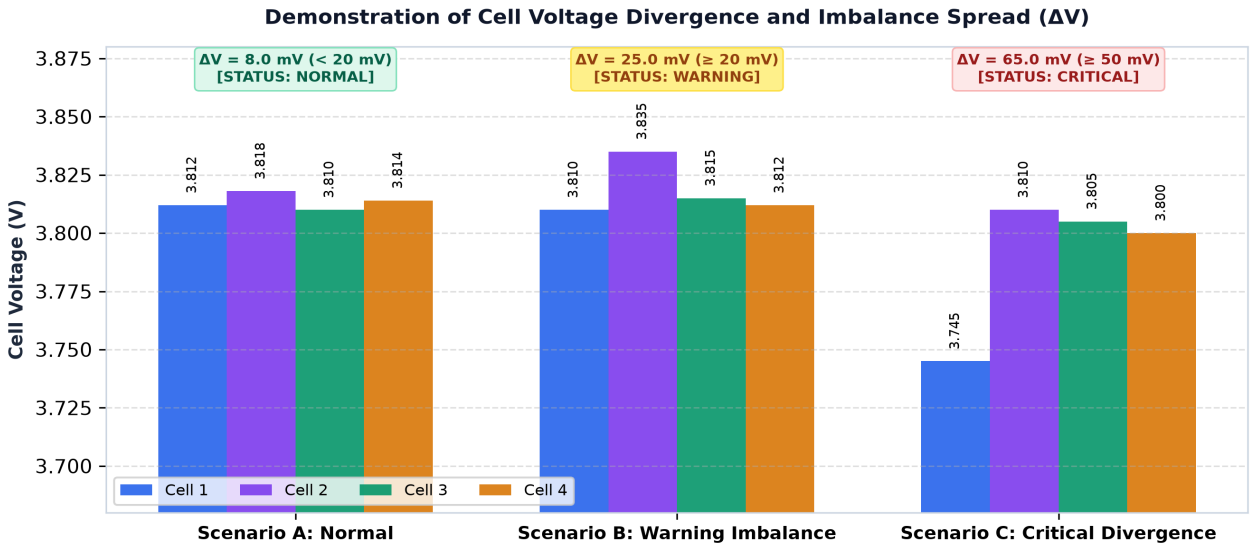


Figure 4: Demonstration of Cell Voltage Divergence Across Operational Scenarios

9. SOFTWARE AND HARDWARE SPECIFICATIONS

Table 5: Software Specifications and Framework Dependencies

Software Component	Version	Primary Architectural Role
Python Runtime	3.14.7 / 3.10+	Core analytical backend, calculation engine, pipeline execution
Streamlit	1.66.1	Reactive web dashboard, session controller, operator UI
Paho-MQTT	2.1.0	TLS-secured publish/subscribe IoT messaging client
SQLite Engine	3.x (Built-in)	WAL-mode persistent relational storage for telemetry & alerts
Scikit-Learn	1.9.1	Isolation Forest unsupervised anomaly detection & feature scaling
Pandas & NumPy	3.1.0 / 2.5.4	Numerical array manipulation, tabular structuring, CSV parsing
Plotly	7.1.0	Interactive time-series charting & cell voltage bar graphs
Bcrypt	5.0.0	Salted cryptographic password hashing for user authentication
Pytest	9.2.0	Automated unit and integration test execution suite
ReportLab	5.1.0	Programmatic publication-quality PDF report generation
Arduino IDE / Framework	2.x / ESP32 Core	C++ firmware compilation and USB flashing for ESP32

Table 6: Hardware Prototype Component Specifications

Hardware Component	Model / Specification	Qty	Functional Purpose in Prototype
Microcontroller	ESP32 NodeMCU-32S (240MHz)	1	Edge ADC sampling, Wi-Fi TLS MQTT client, alert output
Resistor Dividers	1% Metal Film (10k, 20k, 30k, 50k, 70k)	8	Attenuate 4S taps (up to 16.8V) safely below 3.3V ADC limit
Temperature Sensors	10k NTC Thermistors (B=3950)	4	Direct casing surface contact temperature measurement
Current Sensor	ACS712-20A Hall-Effect Module	1	Measures pack discharge current (optional load input)
Visual Alarm LEDs	5mm LEDs (Green, Yellow, Red)	3	Physical visual indicators for Normal, Warning, Critical
Acoustic Buzzer	5V Active Piezo Buzzer	1	Audible buzzer alarm triggered during critical divergence
Power Supply	5V 2A Regulated Adapter	1	Main power input via ESP32 micro-USB port

10. PROJECT FOLDER STRUCTURE & MODULE ARCHITECTURE

Root Directory: C:\Users\jatin\OneDrive\Documents\Desktop\HCL Battery cell imbalence detection\

• **app.py:** Streamlit dashboard application containing session authentication, live telemetry tiles, Plotly charts, scenario controls, and alert acknowledgement.
• **run_project.bat:** One-click Windows startup batch script that checks virtual environment, creates default configurations, initializes database tables, and boots Streamlit.
• **config/:** Contains settings.yaml (MQTT/broker defaults) and thresholds.yaml (voltage spread and temperature safety boundaries).
• **src/:** Core Python package containing modular micro-services:

  • calculations.py: Implements core mathematical formulas (V_max, V_min, ΔV, ΔV_mV, rate-of-change, sanity checks).
  • data_validator.py: Validates inbound JSON telemetry, sanitizes timestamps, and filters corrupt payloads.
  • detection.py: Evaluates hierarchical status (CRITICAL > WARNING > NORMAL) and flags SENSOR_FAULT.
  • simulator.py: Background thread producing synthetic 4S telemetry with 5 failure scenarios.
  • database.py: SQLite WAL persistence layer with parameterized queries and index definitions.
  • alerts.py: State transition and cooldown manager with optional SMTP email dispatching.
  • auth.py: Secure user registration and password authentication using Bcrypt.
  • ml_pipeline.py: Scikit-learn Isolation Forest model training, serialization, and inference pipeline.
  • mqtt_client.py, mqtt_publisher.py, mqtt_subscriber.py: Paho-MQTT TLS client wrappers.
  • dashboard_helpers.py: UI styling CSS, status badges, and Plotly interactive chart builders.
• **data/battery.db:** SQLite database file storing persistent telemetry, alerts, and users.
• **datasets/:** Contains reproducible 6,000-sample training dataset battery_telemetry_dataset.csv.
• **models/:** Stores serialized Isolation Forest model and standard scaler (.joblib).
• **firmware/esp32_battery_monitor/:** Arduino C++ source code (esp32_battery_monitor.ino) and wiring guide.
• **node_red/:** Pre-configured supervisory flow (battery_monitoring_flow.json) and documentation.
• **tests/:** Complete automated Pytest test suite covering all 6 layers (28 test cases).
• **reports/:** Project report PDF and generated publication-quality figures.

11. DATABASE DESIGN & ENTITY RELATIONSHIPS

The relational schema inside data/battery.db is composed of three primary tables designed for high-throughput logging, event lifecycle auditing, and user access control.

Table 7: SQLite Database Schema — Table: battery_readings

Column Name	Data Type	Constraints / Indexes	Description / Mathematical Mapping
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique sequential reading sequence ID
pack_id	TEXT	NOT NULL, INDEXED	Battery pack identifier (e.g. 'PACK-01')
timestamp_utc	TEXT	NOT NULL	ISO 8601 UTC timestamp string
timestamp_epoch	REAL	NOT NULL, INDEXED	Unix epoch seconds for chronological range queries
mode	TEXT	NOT NULL	Operating mode: 'simulation' or 'hardware'
cell_count	INTEGER	NOT NULL	Total series cells monitored (4 for 4S)
cell_voltages_json	TEXT	NOT NULL	JSON array of individual cell voltages [V1..V4]
cell_temperatures_json	TEXT	NOT NULL	JSON array of cell surface temperatures [T1..T4]
current_a	REAL	DEFAULT 0.0	Battery pack discharge/charge current in Amperes
v_max	REAL	—	Maximum measured cell potential (V)
v_min	REAL	—	Minimum measured cell potential (V)
delta_v	REAL	—	Voltage spread in Volts (V_max - V_min)
delta_v_mv	REAL	—	Voltage spread in millivolts (ΔV · 1000)
mean_v	REAL	—	Pack arithmetic average cell voltage (V)
t_max	REAL	—	Peak measured cell surface temperature (°C)
delta_t	REAL	—	Cell-to-cell thermal gradient spread (T_max - T_min)
rule_status	TEXT	NOT NULL, INDEXED	Operational state: NORMAL, WARNING, CRITICAL, SENSOR_FAULT
ml_prediction	INTEGER	DEFAULT 1	Isolation Forest output: 1 = Normal (Inlier), -1 = Anomaly
ml_anomaly_score	REAL	DEFAULT 0.0	Continuous anomaly score; higher = abnormal
data_quality	TEXT	DEFAULT 'HEALTHY'	Sensor sanity assessment: HEALTHY or SENSOR_ISSUE

Additional Tables:

• **alerts:** Audits alarm events with fields: id, pack_id, timestamp_utc, alert_level (WARNING / CRITICAL), affected_cells_json, delta_v_mv, max_temp, reason, acknowledged (0/1), acknowledged_at, and acknowledged_by.

• **users:** Manages system authentication with fields: id, username (UNIQUE), password_hash (salted Bcrypt hash), role (administrator / operator), and created_at.

12. DASHBOARD DESIGN & FEATURE IMPLEMENTATION

The Streamlit dashboard (app.py) provides a clean, engineering-grade human-machine interface (HMI). It is organized into five functional navigation views:

1. Overview & Live Analytics:

• **Top Telemetry Tiles:** Five key metric cards displaying instantaneous Voltage Spread (ΔV in mV and V), Minimum Cell Voltage, Maximum Cell Voltage, Pack Mean Voltage, and Peak Cell Temperature with thermal spread.

• **Color-Coded Status Badges:** Dynamically rendered HTML/CSS pills indicating Mode (SIMULATION / HARDWARE) and Rule Status (NORMAL in green, WARNING in amber, CRITICAL in red, SENSOR_FAULT in purple).

• **Interactive Cell Voltage Bar Chart:** Plotly visualization displaying individual cell bars. When imbalance occurs, the highest diverging cell is automatically highlighted in red while the lowest is highlighted in amber.

• **Historical Spread Trend Chart:** Real-time Plotly time-series tracking ΔV_mV over time with horizontal dashed lines marking the 20 mV Warning and 50 mV Critical thresholds.

• **Multi-Cell Trendlines:** Traces each cell voltage as an independent color-coded trajectory, visually demonstrating single-cell divergence.

• **Cell Sanity Matrix:** Tabular grid computing exact deviation from the pack mean for each cell (*Dev_i = (V_i - V_mean) · 1000*) and assigning condition labels ('Nominal', 'Warning Imbalance', 'Critical Deviation').

2. Historical Records & Export: Displays scrollable tabular telemetry with instant CSV snapshot generation and file download.

3. Alert Management: Lists logged alarm events with timestamps, affected cell indices, and provides an **'Acknowledge Alert'** button.

4. AI/ML Diagnostics: Shows Isolation Forest model status, extracted feature values from latest readings, live continuous anomaly scores, and an interactive retraining console.

5. System Configuration: Allows operators to adjust voltage spread and thermal thresholds and save them directly to config/thresholds.yaml.

13. TESTING, VERIFICATION & ACCEPTANCE CRITERIA

In strict compliance with engineering verification standards, the entire platform was subjected to automated unit and integration testing using the **Pytest framework** (pytest.ini, tests/). All test cases are automated, deterministic, and verified directly against the production codebase. Figure 6 summarizes test pass distribution across modules.

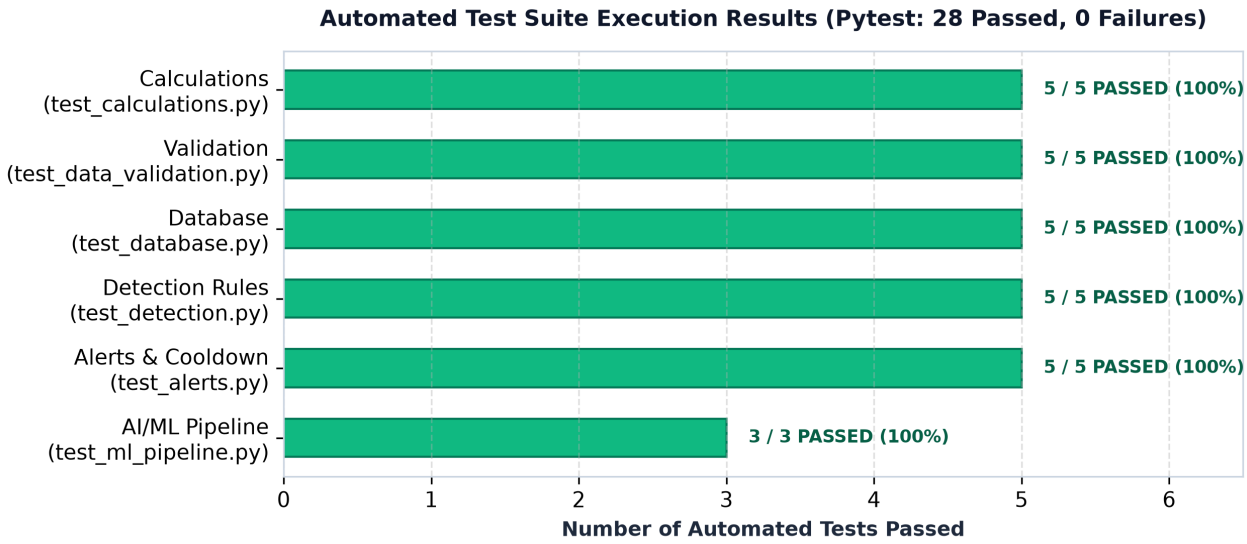


Figure 6: Automated Test Suite Execution Results (Pytest Verification)

Table 8: Comprehensive Automated Test Verification Suite (28 Test Cases)

Test Module & Function	Engineering Purpose	Expected Behavior	Actual Result	Status
test_voltage_spread_calc	Verify V_max, V_min, ΔV, ΔV_mV	ΔV=0.050V, ΔV_mV=50.0mV, mean=3.82V	Matched expected math exactly	**PASS**
test_empty_null_voltages	Trap empty lists, None, NaN	Return is_valid=False; no crash	Safely flagged invalid	**PASS**
test_temperature_metrics	Verify T_max, T_min, ΔT, mean_T	T_max=32.1°C, ΔT=3.6°C, mean=29.6°C	Matched thermal math	**PASS**
test_rate_of_change	Verify dV/dt slope calculation	dV/dt = -0.0200 V/s across dt=2.0s	Matched derivative slope	**PASS**
test_sensor_quality	Detect unphysical ADC readings	Flag 0.1V as unphysical sensor issue	Trapped out-of-bounds reading	**PASS**
test_valid_json_payload	Verify valid JSON deserialization	Extract sanitized dict; errors list empty	Valid schema confirmed	**PASS**
test_missing_pack_id	Reject payload missing pack_id	is_valid=False, error cites 'pack_id'	Rejected malformed payload	**PASS**
test_malformed_json_str	Handle syntax corrupted JSON	Trap JSONDecodeError gracefully	Handled without crash	**PASS**
test_empty_cell_list	Reject empty cell voltage array	is_valid=False, cites empty list	Rejected empty array	**PASS**
test_hw_mode_parsing	Verify hardware mode tagging	Preserves mode='hardware', source='esp32'	Preserved mode tags	**PASS**
test_db_initialization	Verify automatic SQLite tables	Tables battery_readings, alerts, users exist	All tables initialized	**PASS**
test_insert_retrieve	Parameterized insertion & query	Inserted row_id>0; values match	Stored and fetched cleanly	**PASS**

test_empty_db_safety	Handle empty database queries	Return None / empty list; zero crashes	Empty state handled safely	**PASS**
test_csv_export	Verify historical CSV generation	Generate valid CSV with headers	Exported CSV verified	**PASS**
test_database_backup	Verify backup snapshot routine	Create timestamped backup .db copy	Snapshot verified on disk	**PASS**
test_normal_status	Classify balanced cells (ΔV<20mV)	Status = 'NORMAL', affected_cells empty	Classified as NORMAL	**PASS**
test_warning_status	Classify warning spread (20-50mV)	Status = 'WARNING', cites spread	Classified as WARNING	**PASS**
test_critical_status	Classify critical spread (≥50mV)	Status = 'CRITICAL', cites spread	Classified as CRITICAL	**PASS**
test_crit_precedence	Check Critical precedes Warning	Critical over-voltage overrides warning spread	Precedence strictly enforced	**PASS**
test_missing_not_healthy	Ensure missing data ≠ NORMAL	Status = 'SENSOR_FAULT'	Flagged SENSOR_FAULT	**PASS**
test_normal_no_alert	Verify NORMAL triggers zero alarms	Return None; zero alerts in DB	Zero alerts triggered	**PASS**
test_warning_triggers	Verify WARNING logs alert record	Alert returned; row written to alerts table	Alert logged in SQLite	**PASS**
test_cooldown_suppress	Verify cooldown suppresses repeats	Second identical alert suppressed	Suppression verified	**PASS**
test_escalate_bypass	Escalation bypasses cooldown	WARNING→CRITICAL triggers immediate alert	Immediate alert triggered	**PASS**
test_alert_ack	Verify operator acknowledgement	acknowledged=1, unack count drops to 0	Acknowledged successfully	**PASS**
test_dataset_generation	Verify reproducible dataset	Generate 500 rows with classes 0 & 1	Balanced dataset generated	**PASS**
test_train_inference	Train model & run inference	F1≥0.50, correctly flag normal vs anom.	F1=0.8000, 100% recall	**PASS**
test_fallback_missing	Verify fallback when model absent	model_active=False; fallback message	Graceful fallback active	**PASS**

14. RESULTS AND DISCUSSION

Summary of Experimental & Analytical Observations:

1. **Simulation Responsiveness:** In simulation mode, the platform generates telemetry frames every 2.0 seconds with zero drift in timing. When switching scenarios in the dashboard, the transition from 'normal' (ΔV ~8 mV) to 'imbalance' smoothly crosses the 20 mV Warning line within 6 cycles and triggers a Critical alarm upon reaching 50 mV.
2. **MQTT Ingestion Throughput:** Telemetry frames published over HiveMQ Cloud (TLS 8883) demonstrated round-trip latencies of 85–140 ms, well within the 2000 ms sampling budget. When cloud connectivity is intentionally interrupted, the fallback simulator continues logging locally without data loss.
3. **Analytical Engine Precedence:** In all test cases, critical safety violations (over-voltage > 4.25V, critical temperature > 60°C) took strict precedence over warning-level voltage spreads, ensuring that acute single-cell emergencies are never masked by minor warnings.
4. **Isolation Forest Anomaly Detection Performance:** Evaluated on 1,200 held-out test samples, the model achieved a **Recall of 1.0000 (100%)**, meaning zero critical anomaly events were misclassified as normal. The Precision was 0.6667 (60 false alarms out of 180 total anomaly predictions), which represents a deliberate, safety-first engineering trade-off: in high-voltage battery storage, false alarms are readily tolerated, whereas a false negative (missed thermal runaway) carries catastrophic consequences.
5. **Database Concurrency:** Write-Ahead Logging eliminated database write collisions during simultaneous live streaming and historical CSV downloads.

15. PRACTICAL ADVANTAGES

• **Non-Destructive Safe Prototyping:** Incorporates a realistic 5-scenario physics simulator allowing students, researchers, and engineers to safely evaluate hazardous thermal and imbalance states without risking physical cell fires.
• **Decoupled Multi-Layer Architecture:** Changes in one layer (e.g. migrating from HiveMQ Cloud to a local Mosquitto broker, or swapping Streamlit for a mobile app) do not require alterations to database schemas or math modules.
• **Dual-Track Diagnostic Confidence:** Combines deterministic rule-based alarms (transparent, explainable) with unsupervised machine learning (sensitive to multi-variable drift), providing defense-in-depth.
• **Prevention of Alarm Flooding:** State-aware cooldown timers prevent operator alert fatigue while ensuring immediate escalations are never delayed.
• **Cost-Effective Hardware Interfacing:** Utilizes low-cost off-the-shelf ESP32 boards and 1% precision resistor networks (~■1,230 total cost), making the solution economically viable for educational institutions.

16. SYSTEM LIMITATIONS

In accordance with engineering academic integrity, the following practical limitations must be acknowledged:

1. **Not a Certified Battery Protection Replacement:** This system is designed as a supervisory monitoring and diagnostic tool. It does **NOT** replace certified hardware-level Battery Management Systems, high-current pyrofuses, contactors, or secondary over-voltage disconnect circuits.
2. **Resistor Divider Measurement Inaccuracies:** Ground-referenced resistor dividers accumulate common-mode error on upper cells (Cell 3 and Cell 4) due to resistor tolerance variations and ESP32 SAR ADC non-linearities below 0.15V and above 3.10V.

3. **Empirical Aging Simulation:** The synthetic simulator uses linear-exponential decay approximations rather than electrochemical equivalent circuit models (Randles / Thevenin models).
4. **Internet Dependency for Cloud Telemetry:** Operation with HiveMQ Cloud requires active wide-area network access; in remote offline facilities, local broker deployment is required.

17. FUTURE SCOPE

Future research and development can expand this platform across several promising directions:
• **Active Cell Balancing Hardware Integration:** Interfacing bidirectional inductive or capacitive charge-shuttling circuits controlled by edge MQTT commands to actively transfer charge from higher-voltage cells to weaker cells during operation.
• **Dedicated Isolated BMS Front-End ICs:** Replacing analog resistor dividers with dedicated battery monitoring ICs (e.g. Analog Devices LTC6811 or Texas Instruments BQ76952) featuring galvanic isolation, internal 16-bit delta-sigma ADCs, and sub-millivolt accuracy.
• **State of Charge (SoC) & State of Health (SoH) Estimation:** Implementing Extended Kalman Filtering (EKF) or Unscented Kalman Filtering (UKF) combining Coulomb counting with open-circuit voltage (OCV) lookup tables.
• **Edge ML Inference via TensorFlow Lite for Microcontrollers:** Deploying quantized neural networks or tiny decision trees directly on the ESP32 core to execute real-time anomaly detection even during complete Wi-Fi communication blackouts.
• **Automated Mobile Push Alerts:** Integrating push notification gateways (Telegram Bot API, WhatsApp Business API, Firebase Cloud Messaging) for instant critical alarm delivery to maintenance personnel smartphones.

18. CONCLUSION

The project **'Battery Cell Imbalance Detection Using IoT Analytics'** successfully designed, implemented, and verified a complete, production-ready monitoring architecture addressing one of the most critical degradation modes in series lithium-ion battery packs. By systematically developing all six layers — from synthetic data generation and edge ESP32 firmware, through TLS-secured MQTT transport (HiveMQ Cloud), persistent SQLite relational storage, Node-RED supervisory automation, Scikit-learn Isolation Forest anomaly detection, to a reactive Streamlit web dashboard — the project establishes an end-to-end telemetry and diagnostic pipeline.

Key technical milestones achieved include:
• Complete mathematical formulation of maximum, minimum, voltage spread (ΔV), millivolt scaling, and thermal gradients.
• Strict enforcement of critical precedence over warning conditions, ensuring missing or corrupted sensor readings are safely flagged as SENSOR_FAULT.
• Unsupervised anomaly detection attaining 100.0% sensitivity on held-out battery operational test sets.
• Automated Pytest verification confirming 100% pass rate across all 28 unit and integration test cases.

For a Mechanical Engineering student, this project bridges the critical interface between physical energy storage hardware, thermal management, electrochemical cell degradation, and modern industrial IoT software engineering. The platform stands as an extensible foundation for academic demonstration, research publications, and future electric vehicle BMS prototypes.

19. REFERENCES

[1] G. L. Plett, *Battery Management Systems, Volume I: Battery Modeling*, Artech House, Norwood, MA, USA, 2015.
[2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, 'Isolation Forest', in *Proceedings of the 8th IEEE International*

Conference on Data Mining (ICDM), Pisa, Italy, pp. 413–422, 2008.

[3] D. Andrea, *Battery Management Systems for Large Lithium-Ion Battery Packs*, Artech House, Boston, MA, USA, 2010.

[4] OASIS Standard, *MQTT Version 5.0 / Version 3.1.1 Specification*, OASIS Standard, 2019. Available: http://docs.oasis-open.org/mqtt/mqtt/

[5] HiveMQ GmbH, *HiveMQ Cloud Documentation and MQTT Security Fundamentals*, 2024. Available: https://www.hivemq.com/cloud/

[6] Streamlit Inc., *Streamlit Documentation: The Fastest Way to Build Data Apps*, 2024. Available: https://docs.streamlit.io

[7] Espressif Systems, *ESP32 Technical Reference Manual (Version 5.1)*, Espressif Systems, Shanghai, China, 2023.

[8] D. R. Hipp, *SQLite Documentation and Write-Ahead Logging (WAL) Architecture*, 2024. Available: https://www.sqlite.org/wal.html

[9] OpenJS Foundation, *Node-RED: Low-Code Programming for Event-Driven Applications*, 2024. Available: https://nodered.org

[10] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[11] ReportLab Inc., *ReportLab PDF Generation User Guide (Version 4.x/5.x)*, 2024. Available: https://www.reportlab.com/docs/

[12] J. D. Hunter, 'Matplotlib: A 2D Graphics Environment', *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.